Implementation of Fixed-Point Division Using Newton–Raphson Method

Aslin Garvasis
EE24BTECH11008

1 Introduction

Division in digital hardware is significantly more expensive than multiplication. A common architectural strategy is to compute the reciprocal of the denominator using an iterative scheme and then multiply by the numerator:

$$\frac{A}{D} = A \cdot \frac{1}{D}$$

In this work, the reciprocal is computed using the Newton–Raphson method under Q2.30 fixed-point arithmetic. MATLAB is used as a numerical reference and Verilog is used for hardware realization.

2 Derivation of Newton–Raphson Method

Let $f \in C^2[a, b]$ and suppose p is a simple root of f, i.e.,

$$f(p) = 0, \quad f'(p) \neq 0$$

Let p_0 be an approximation to p such that $|p - p_0|$ is small.

2.1 Taylor Expansion

Expanding $f(x)$ about $x = p_0$ and evaluating at $x = p$:

$$f(p) = f(p_0) + (p - p_0)f'(p_0) + \frac{(p - p_0)^2}{2} f''(\xi)$$

for some ξ between p and p_0.
Since $f(p) = 0$,

$$f(p_0) + (p - p_0)f'(p_0) + \frac{(p - p_0)^2}{2} f''(\xi) = 0$$

If p_0 is close to p, the quadratic term is negligible, giving

$$f(p_0) + (p - p_0)f'(p_0) \approx 0$$

Solving for p:

$$p \approx p_0 - \frac{f(p_0)}{f'(p_0)}$$

Thus the Newton iteration is

$$\boxed{p_{k+1} = p_k - \frac{f(p_k)}{f'(p_k)}}$$

2.2 Quadratic Convergence

Retaining the second-order term shows that the error satisfies

$$e_{k+1} \approx Ce_k^2$$

where $e_k = p_k - p$. Thus convergence is quadratic.

3 Application to Reciprocal Computation

To compute $1/D$, define

$$f(x) = \frac{1}{x} - D$$

Then

$$f'(x) = -\frac{1}{x^2}$$

Substituting into Newton iteration:

$$x_{k+1} = x_k - \frac{\frac{1}{x_k} - D}{-\frac{1}{x_k^2}}$$

Simplifying:

$$x_{k+1} = x_k(2 - Dx_k)$$

This multiplicative form is well suited for hardware implementation.

4 Normalization

For improved numerical stability, the denominator is scaled such that

$$0.5 \leq D_{norm} < 1$$

If

$$D_{norm} = D \cdot 2^s$$

then

$$\frac{1}{D} = \frac{1}{D_{norm}} \cdot 2^s$$

The final result is rescaled accordingly.

5 Fixed-Point Representation

All computations are performed in Q2.30 format:

$$\text{Real value} = \frac{\text{Integer representation}}{2^{30}}$$

Multiplication rule:

$$(a \cdot b)_{fixed} = (a \cdot b) >> 30$$

The resolution of this format is

$$2^{-30} \approx 9.31 \times 10^{-10}$$

This determines the achievable precision limit.

6 Minimax Linear Initial Approximation

6.1 Problem Formulation

After normalization,

$$0.5 \leq d < 1$$

We approximate

$$\frac{1}{d}$$

using a linear function

$$x_0(d) = a - bd$$

such that the maximum absolute error over $[0.5, 1]$ is minimized.

6.2 Error Function

$$E(d) = \frac{1}{d} - a + bd$$

Using equioscillation theory and solving the resulting system yields

$$a = \frac{48}{17}, \qquad b = \frac{32}{17}$$

Thus,

$$x_0(d) = \frac{48}{17} - \frac{32}{17}d$$

Numerically,

$$x_0(d) \approx 2.823529 - 1.882353d$$

This provides bounded worst-case error and rapid convergence.

7 Lookup Table Based Initial Approximation

In addition to linear minimax approximation, a common hardware-oriented approach for generating the initial estimate in Newton–Raphson iteration is the use of a lookup table (LUT).

7.1 Motivation

Newton–Raphson exhibits quadratic convergence provided that the initial estimate is sufficiently close to the true reciprocal. A lookup table provides a fast and accurate way to obtain such an estimate with minimal runtime computation.

7.2 Step 1: Normalization

The denominator is first normalized such that

$$0.5 \leq d < 1$$

If the original denominator is D, then

$$d = D \cdot 2^s$$

for some integer shift s chosen to bring d into the required interval.
This ensures that the reciprocal lies in the bounded range

$$1 \leq \frac{1}{d} < 2$$

which simplifies approximation.

7.3 Step 2: Interval Partitioning

The interval $[0.5, 1)$ is divided into 2^n equal subintervals.
For example, using the top 6 bits of the normalized denominator,

$$2^6 = 64 \text{ segments}$$

Each segment has width

$$\Delta = \frac{1 - 0.5}{2^n}$$

7.4 Step 3: Lookup Table Generation

For each segment midpoint d_i, a precomputed approximation of the reciprocal is stored:

$$\text{LUT}[i] = \text{round}\left(\frac{1}{d_i} \cdot 2^F\right)$$

where F is the number of fractional bits in the fixed-point representation.
This table is generated offline and stored in memory.

7.5 Step 4: Index Computation

At runtime, the table index is computed using the most significant bits of the normalized denominator:

$$\text{index} = \left\lfloor \frac{d - 0.5}{0.5} \cdot 2^n \right\rfloor$$

In fixed-point hardware, this can be implemented efficiently using subtraction and right-shift operations.

7.6 Step 5: Initial Estimate Retrieval

The initial estimate is obtained as

$$x_0 = \text{LUT}[\text{index}]$$

Since the LUT stores reciprocal values corresponding to narrow intervals, the initial approximation error is small.

7.7 Step 6: Newton Refinement

The Newton iteration is then applied:

$$x_{k+1} = x_k(2 - dx_k)$$

Because the LUT provides a close starting point, typically only 2–3 iterations are required to reach fixed-point precision limits.

7.8 Hardware Trade-offs

- LUT-based initialization reduces iteration count.
- Requires memory resources (BRAM or distributed LUTs).
- Linear minimax approximation avoids memory usage but requires a multiplier.
- The choice depends on FPGA resource availability and design constraints.

8 Algorithm Summary

1. Normalize denominator
2. Compute linear initial estimate
3. Perform 5 Newton iterations
4. Rescale reciprocal
5. Multiply by numerator

9 Test Cases

The following prime denominator cases were tested:

$$\frac{15}{23}, \quad \frac{12}{37}, \quad \frac{19}{101}, \quad \frac{8}{59}, \quad \frac{27}{83}$$

10 Simulation Results

MATLAB Reference Output

```
Test: 15 / 23
Result   : 0.652173913084
Expected : 0.652173913043
Error    : 4.049228e-11

Test: 12 / 37
Result   : 0.324324323796
Expected : 0.324324324324
Error    : 5.285885e-10

Test: 19 / 101
Result   : 0.188118811697
Expected : 0.188118811881
Error    : 1.844203e-10

Test: 8 / 59
Result   : 0.135593219660
Expected : 0.135593220339
Error    : 6.787605e-10

Test: 27 / 83
Result   : 0.325301204808
Expected : 0.325301204819
Error    : 1.122075e-11
```

Verilog Hardware Simulation

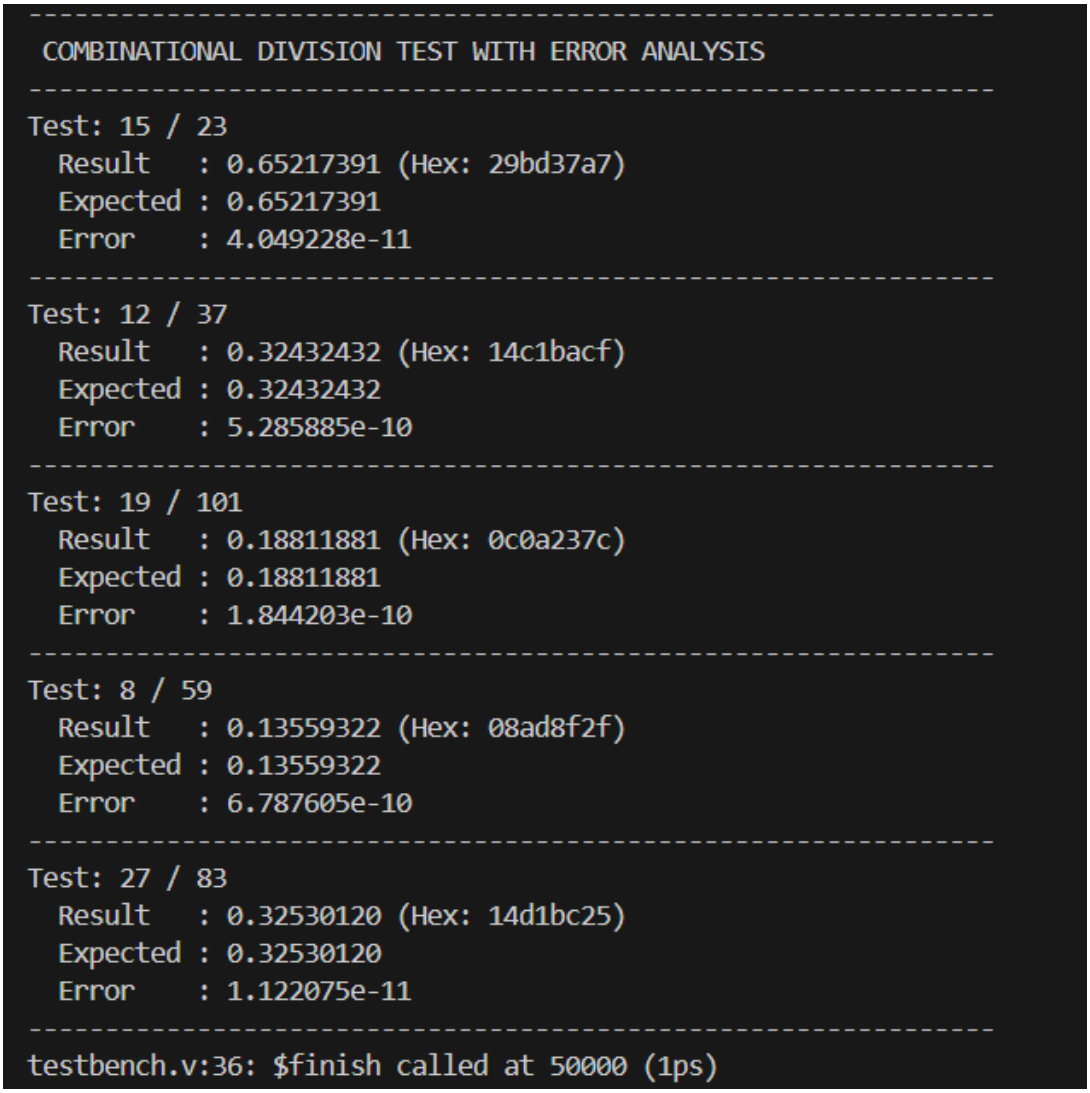

Figure 2: Verilog simulation output

11 Discussion

The Verilog results exact match the MATLAB reference outputs. The observed absolute error is on the order of 10^{-10}, which lies within the Q2.30 quantization resolution.

Small deviations arise due to truncation and fixed-point rounding effects.

12 Conclusion

A fixed-point Newton–Raphson divider was successfully implemented and verified. The use of normalization and a minimax-based linear initial approximation enables rapid quadratic convergence while maintaining hardware efficiency.